Based on your LMC 3402 Generative Art assignment slides, I recommend keeping your project simple, with one main HTML file, a context file, and separate JavaScript files only when necessary.

1. Recommended file structure

```
quayola-generative-art/
│
├── AGENTS.md          # Instructions for Codex
├── context.md         # Artistic intent, rules, seed log, milestones
├── index.html         # Main webpage and canvas
├── .gitignore         # Excludes generated files
│
├── geometry.js        # Generates shapes and transformations
├── renderer.js        # Draws geometry on the screen
├── svg-export.js      # Exports geometry as A3 SVG
│
└── exports/
    ├── seed-48213.png
    └── seed-48213.svg
```

Important: The three JavaScript files are optional initially. Your professor recommends beginning with a self-contained `index.html` and splitting the JavaScript into separate files when the code exceeds approximately 300 lines.

2. Purpose of each file

File	Purpose
`AGENTS.md`	Persistent Codex instructions. Tells the agent to read `context.md`, plan before coding, and update progress. Optional addition for using Codex.
`context.md`	Main source of truth. Contains your artist intent, generative rules, randomness constraints, seed log, file list, and milestone checklist.
`index.html`	Main executable webpage. Contains the canvas, controls, and loads JavaScript files if separated.

`geometry.js`	Generates geometry using a deterministic random seed and normalized 0–1 coordinates.
`renderer.js`	Renders the generated geometry onto the canvas for screen display and PNG export.
`svg-export.js`	Converts the same generated geometry into stroke-only SVG paths for the A3 pen plotter.
`exports/`	Stores PNG and SVG artwork generated using identical seeds.
`.gitignore`	Prevents `exports/`, `.env`, and `.DS_Store` from being committed to Git.

3. Requirements from the assignment

Your implementation must support:

- Vertical 4K canvas: 2160 × 3840 pixels, scaled to fit the browser without distortion.
- A3 SVG export: 297 × 420 mm, using strokes only, with no fills.
- Seeded randomness: The same seed must reproduce the same composition.
- Normalized geometry: Generate compositions in 0–1 coordinates rather than fixed pixel coordinates.
- PNG and SVG export: Both must represent the same seeded artwork.
- Keyboard controls: `R` = generate new seed, `S` = save PNG, `P` = export SVG.
- Animation loop: `requestAnimationFrame`, targeting 60 FPS, with an FPS and seed display.
- Documentation: Update `context.md` as implementation progresses.

4. How to organize context.md

Keep everything in one file with these sections:

```
# Workspace Blueprint

## Project Overview
- Concept
```

- Artist intent
- Inspirations (Quayola's Strata)
- Outputs and technologies

System Rules
- Geometric generation rules
- Randomized parameters and ranges
- Fixed constraints
- Composition and distortion rules

Architecture Rules
- All randomness comes from one seeded RNG
- Geometry uses normalized coordinates
- Rendering and export share geometry
- Split scripts after approximately 300 lines

File Structure
- List of project files and their responsibilities

Seed Log
- Seed number
- Parameters
- Visual observations
- Keep/reject decisions

Milestones
- [ ] M1: Scaffold + PNG/SVG export
- [ ] M2: Generative rules
- [ ] M3: 4K screen refinement
- [ ] M4: A3 pen plotter refinement
- [ ] M5: Final seed curation

The seed log belongs inside `context.md`, as the slides explicitly recommend.

5. How to divide Codex conversations

Use one chat per milestone, with every chat reading the same `context.md`.

Chat	Focus

Chat 1	Build basic HTML, canvas, seeded RNG, PNG/SVG export
Chat 2	Implement your Quayola-inspired geometric generation rules
Chat 3	Optimize visual appearance on the vertical 4K screen
Chat 4	Refine stroke-only SVG export for the A3 pen plotter
Chat 5	Explore seeds, finalize parameters, curate artwork

After each milestone, have Codex update `context.md` and make a Git commit after reviewing the changes.

6. Relationship between HTML and JavaScript

index.html

Loads the application and its JavaScript

geometry.js

Generates deterministic shapes and paths

renderer.js

Canvas / PNG

svg-export.js

A3 SVG

The crucial design principle is that geometry is generated once and shared between the canvas renderer and SVG exporter, rather than implementing two independent drawing algorithms.

Bottom line: Start with `AGENTS.md`, `context.md`, `index.html`, `.gitignore`, and `exports/`. Add the three JavaScript files when the code becomes complex. Keep one repository and use separate Codex conversations for each of your five milestones.

This follows the assignment's file structure and technical requirements while adding a lightweight Codex workflow.